paytag
Shopping Made Effortless

FullStack Developer Task

1. Introduction

Hi, and thank you for your interest in joining our team!

This task is an opportunity for you to showcase your habits of work, your way of thinking about an existing codebase, and how cleanly you can add new behavior without disturbing what's already there.

The recruitment process goes as follows:

1. This task is for you to make at home, with your environment
2. The code of the task needs to be sent 7 days after receiving it
3. At a scheduled meeting, you will be invited to meet us at our offices
4. You will walk us through your work, the design choices you made, and the alternatives you considered

2. Summary

As our **FullStack Developer**, you are working on **FastCheckout** — a Windows client (WinForms, .NET 8) that connects to an RFID reader, runs inventory, and surfaces the scanned tags in real time.

In production, the reader is a Zebra / Symbol fixed reader on the store's local network. For this exercise you will be running against a built-in **simulator** — no hardware is required.

2.1 Composition of the App

- **Local Application:** Windows desktop app (WinForms, .NET 8) hosting the form, the reader controller, and a global keyboard hook.
- **IoT Reader:** Ships as a precompiled DLL (`lib/FastCheckout.Reader.dll`). Same public API as the production reader, so code you write here would work unchanged against the real hardware.
- **Simulator:** Built into the DLL — emits realistic EPC strings on a timer once inventory is started.

2.2 What's Already Implemented

- The form connects to the reader on startup, marshals all reader events to the UI thread, and shuts the reader down cleanly on exit.

- The reader DLL handles connect, inventory start/stop, and emits a `TagScanned` event for each tag.
- The form already renders incoming tags (newest-first, with `HH:mm:ss.fff` timestamps, capped at 200 rows) and updates the count label.
- A `GlobalKeyboardHook` class is scaffolded with all the P/Invoke signatures, constants, delegate type, fields, and the `KeyPressed` event in place — `Install()` and `Dispose()` are the stubs.

2.3 App Flow

1. The app launches; the form opens and the reader simulator connects automatically — **Reader: Connected** appears in green.
2. The user presses **S** anywhere on their system (e.g. while focused on a POS field in another app) — inventory **starts**, and simulated tags begin streaming in at ~700 ms intervals.
3. Pressing **S** again **stops** inventory; the tag stream halts.
4. Clicking the window's **X** hides the form to the **system tray**; left-clicking the tray icon restores it.
5. The tray menu's **Close** entry shuts down the reader cleanly and exits.

3. Your Task

1. Make pressing **S** anywhere on the system toggle the reader's inventory.
2. Decide your approach. The `GlobalKeyboardHook` class is scaffolded as a hint — but if you'd prefer `RegisterHotKey`, `Application.AddMessageFilter`, or a different mechanism, that's a fair choice. Be ready to defend it.
3. **Plan in writing before coding**: which existing files you'll read first, where your changes land, what behaviors must keep working, and which alternatives you considered.
4. Commit at logical checkpoints — small, named commits beat one giant "done" commit.

Behaviors to Preserve

Before declaring the task done, verify each of these still works exactly as it did before your changes:

- ☐ App launches and connects on startup — status flips to **Reader: Connected** (green).
- ☐ Tags display newest-first with `HH:mm:ss.fff` timestamps; the list caps at 200 rows.
- ☐ The tag-count label updates as tags arrive.
- ☐ Tray icon: left-click toggles form visibility; right-click shows **Show RFID Controller** + **Close**.
- ☐ Clicking the window's **X** hides the form (doesn't exit); the tray **Close** menu exits cleanly.
- ☐ Reader shuts down on exit — no orphaned process, no errors in the log.
- ☐ Typing **S** into Notepad / your terminal / any other field still produces an "s" — your toggle must not swallow the keystroke.

Notes:

1. The code needs to be clean, structured, and easy to follow.
2. Suggested time budget: **~2 hours of focused work** — 15 min reading + planning, 60–75 min on the main task, ~30 min on one follow-up of your choice.
3. If you are using AI to do this task, **provide a full transcript of the AI conversation** (link or text export). Make sure you understand the code before you start working on the task.
4. Too easy? Pick **one** of the follow-ups below — implement it if there's time, otherwise come ready to design it out loud:
   a. **Configurable toggle key** — move the hardcoded `S` into `Configuration/config_computer.json`, defaulting to `S` if missing or unparseable.
   b. **Wire the existing EPC decoder** — `RfidEpcParser.ExtractBarcodeFromRfid` is in the project but unused. Show the decoded barcode alongside the raw EPC; handle `null` gracefully.
   c. **Per-session tag deduplication** — show each `TagID` at most once per inventory session; reset on stop.
   d. **Auto-reconnect on disconnect** — watch for `ConnectionChanged(false)` and retry with backoff. The simulator never disconnects, so this is mostly a design conversation.

4. Material Provided

1. The **codebase** — zipped, including the `.sln`, source files, the reader simulator DLL, and this brief.
2. The **reader simulator** — `lib/FastCheckout.Reader.dll`. Runs locally, no hardware required.
3. This **brief** — the document you're reading.